



Walmart Sales Forecasting

DASHBOARD & MODELING DOCUMENTATION

Forecasting weekly sales for 45 Walmart stores, 12 weeks ahead, using a seasonal time-series model — wrapped in an interactive Streamlit dashboard.

Python

SARIMA

Streamlit

45 Stores

Author: Siddu · B.Sc. Mathematics, Statistics & Data Science

GitHub: github.com/sidducv0528/Walmart-Sales-Forecasting

Contact: sidducv0528@gmail.com

1. Project Overview

Objective

Forecast weekly sales for the next 12 weeks across all 45 Walmart stores, and evaluate how reliable that forecast is on a **per-store basis** — rather than assuming a single model performs equally well everywhere.

Dataset

Property	Value
Total records	6,435 weekly records
Stores	45
Weeks per store	143 (Feb 2010 – Oct 2012)
Columns	Store , Date , Weekly_Sales , Holiday_Flag , Temperature , Fuel_Price , CPI , Unemployment

Tech Stack

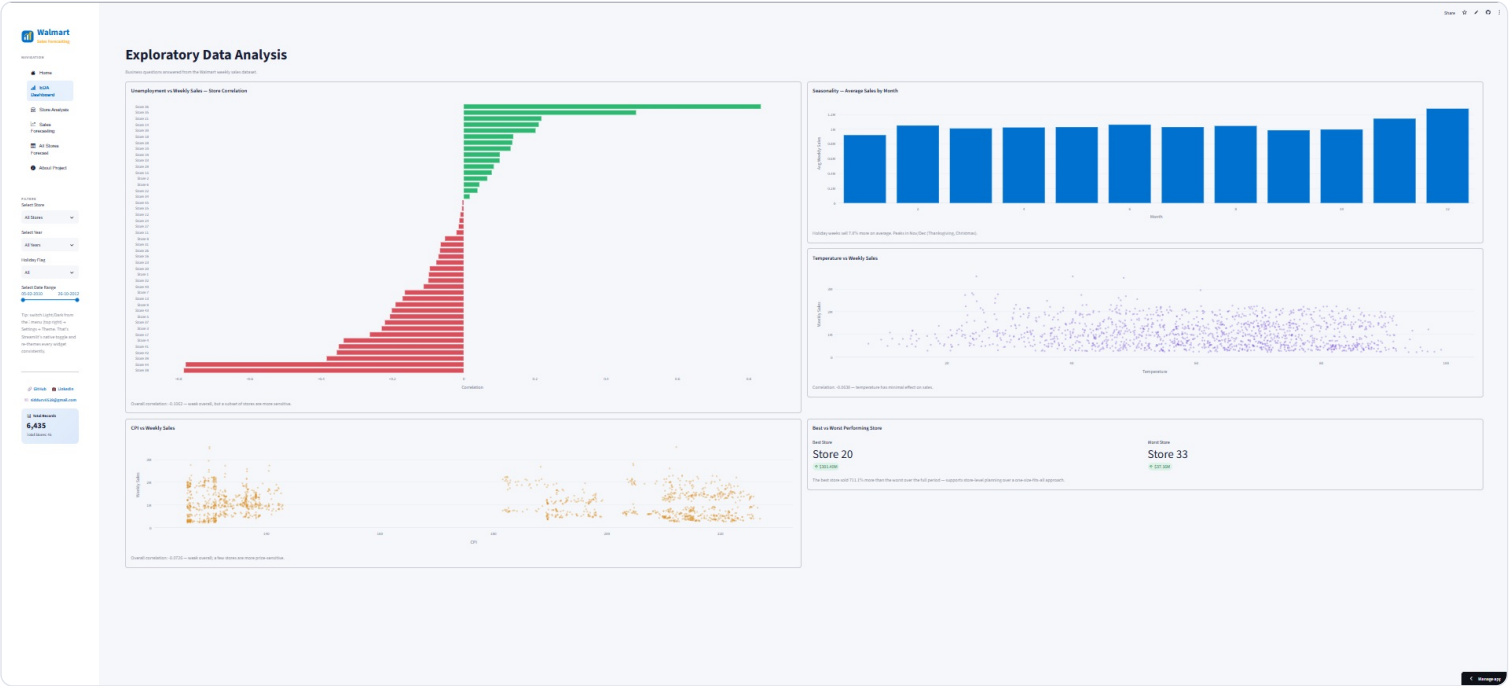
Layer	Tools
Data handling	pandas, numpy
Modeling	statsmodels (SARIMAX class, used as SARIMA)
Dashboard	Streamlit, Plotly, streamlit-option-menu
Deployment	Streamlit Community Cloud, GitHub

2. Exploratory Data Analysis

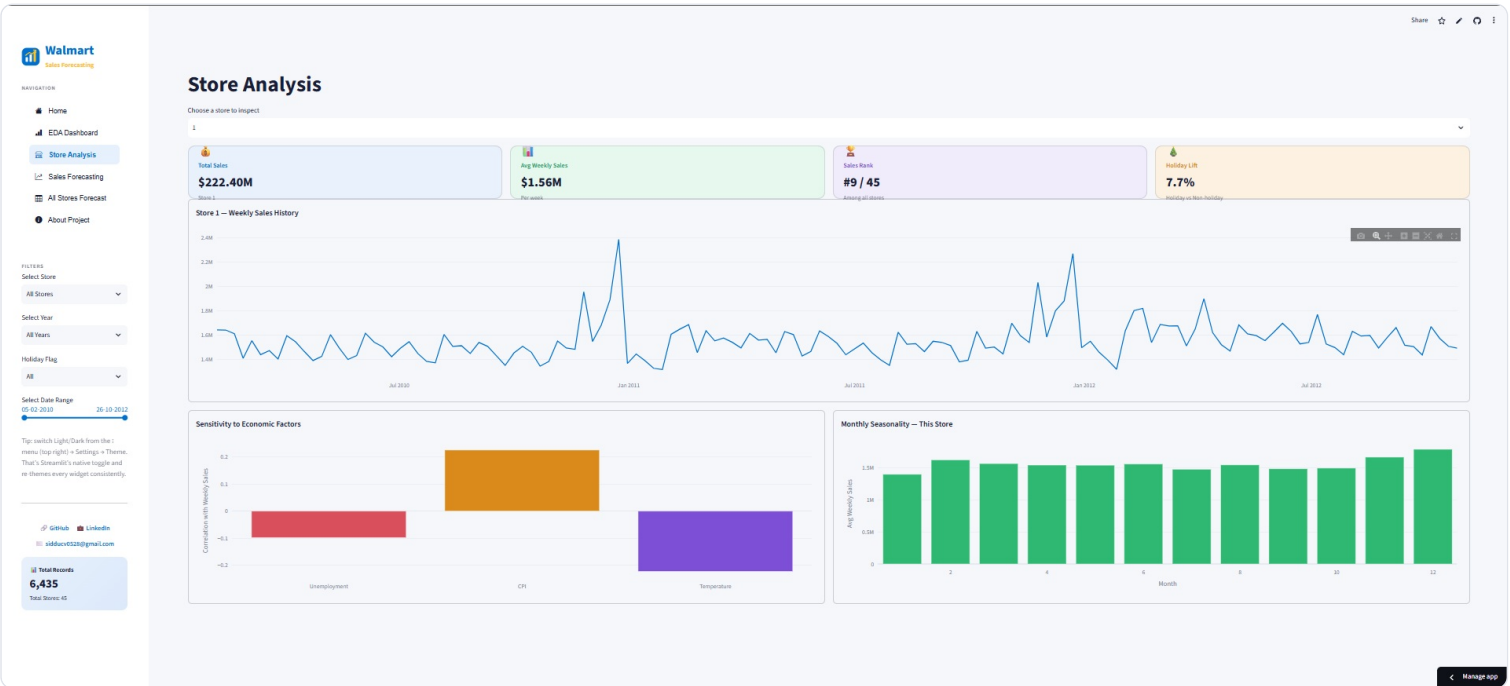
Before modeling, the dataset was explored across several dimensions to understand what actually drives weekly sales, and to inform decisions made later in the modeling stage.

Key Findings

- **Seasonality is the dominant signal.** Sales peak sharply in November–December (Thanksgiving/Christmas), with holiday weeks averaging roughly 7–8% higher sales than non-holiday weeks.
- **Unemployment and CPI have weak overall correlation with weekly sales** — both under 0.1 in magnitude at the aggregate level — though a handful of individual stores show noticeably stronger sensitivity than others.
- **Temperature has minimal effect on sales** (correlation ≈ -0.02).
- **Large performance gap between best and worst stores** — the top-selling store moved over 7× the volume of the bottom store across the full period, which is the core argument for store-level rather than chain-wide planning.



EDA Dashboard — unemployment correlation by store, seasonality, temperature and CPI relationships, best vs. worst store comparison



Store Analysis — per-store deep dive: sales rank, holiday lift, economic sensitivity, monthly seasonality

3. Modeling Methodology

Model

SARIMA (1,1,1)(1,1,1,52) — fit independently for each of the 45 stores. The seasonal order of 52 captures the yearly weekly cycle directly, which is what drives the holiday-season spike observed in nearly every store.

A note on model naming — SARIMA vs. SARIMAX:

The dataset includes `Holiday_Flag` , `CPI` , and `Unemployment` as candidate exogenous regressors. These were evaluated and deliberately **not** included as exogenous inputs, for two reasons backed directly by the EDA above:

- Holiday effects are already captured by the seasonal `(..., 52)` term — adding `Holiday_Flag` separately would be largely redundant.
- CPI and Unemployment are macroeconomic indicators that move slowly month-to-month and showed weak correlation with weekly sales at the store level — they are unlikely to meaningfully improve a 12-week-ahead forecast.

This is implemented using statsmodels' `SARIMAX` class (which supports exogenous inputs), but no exogenous variables are passed — functionally this is a **seasonal ARIMA (SARIMA)** model, not true SARIMAX.

Evaluation Approach

The last 12 weeks of each store's history are held out as a test set. The model is trained on all prior weeks, then forecasts are compared against the actual held-out values using three metrics, computed per store:

- **MAE** — Mean Absolute Error
- **RMSE** — Root Mean Squared Error
- **MAPE** — Mean Absolute Percentage Error (used for the rating below)

MAPE	Rating
< 10%	Excellent
< 20%	Good
≥ 20%	Needs Improvement

4. Results

AVG MAPE

3.66%

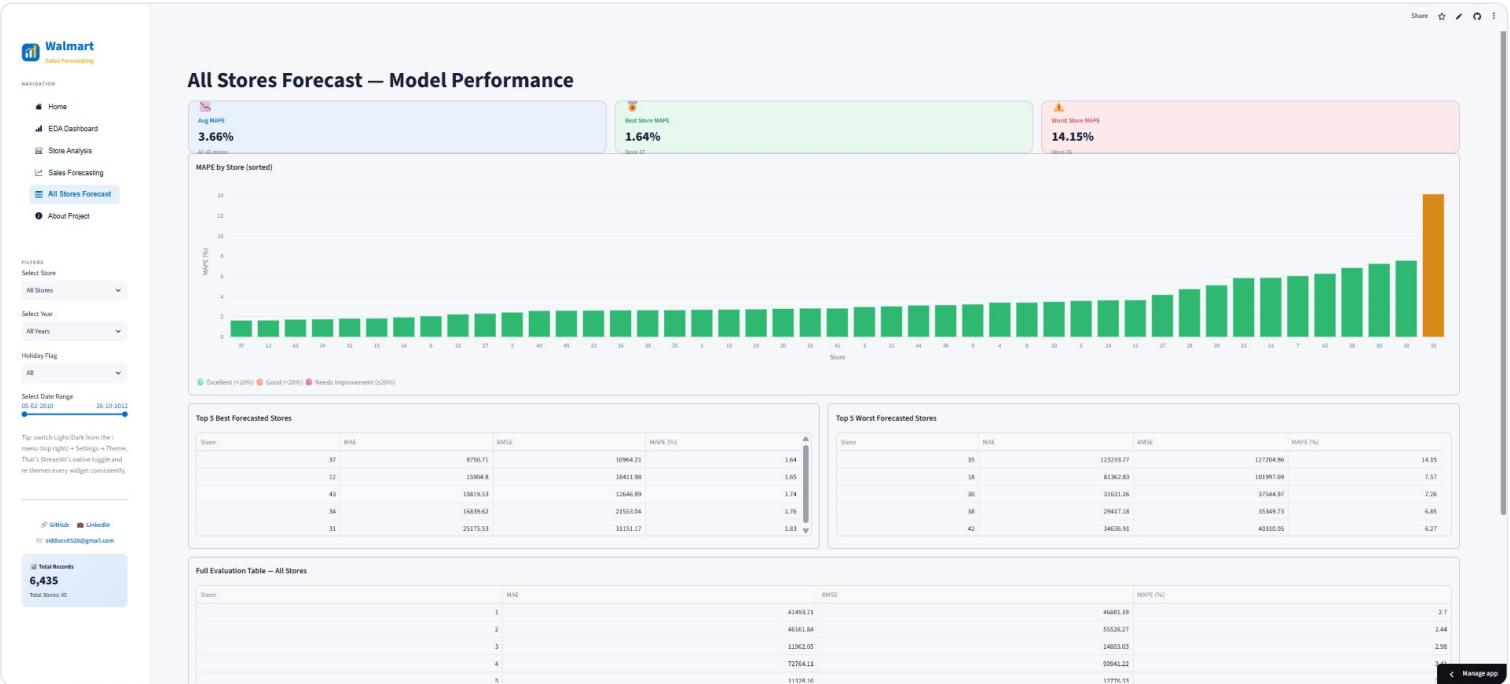
BEST STORE MAPE

1.64%

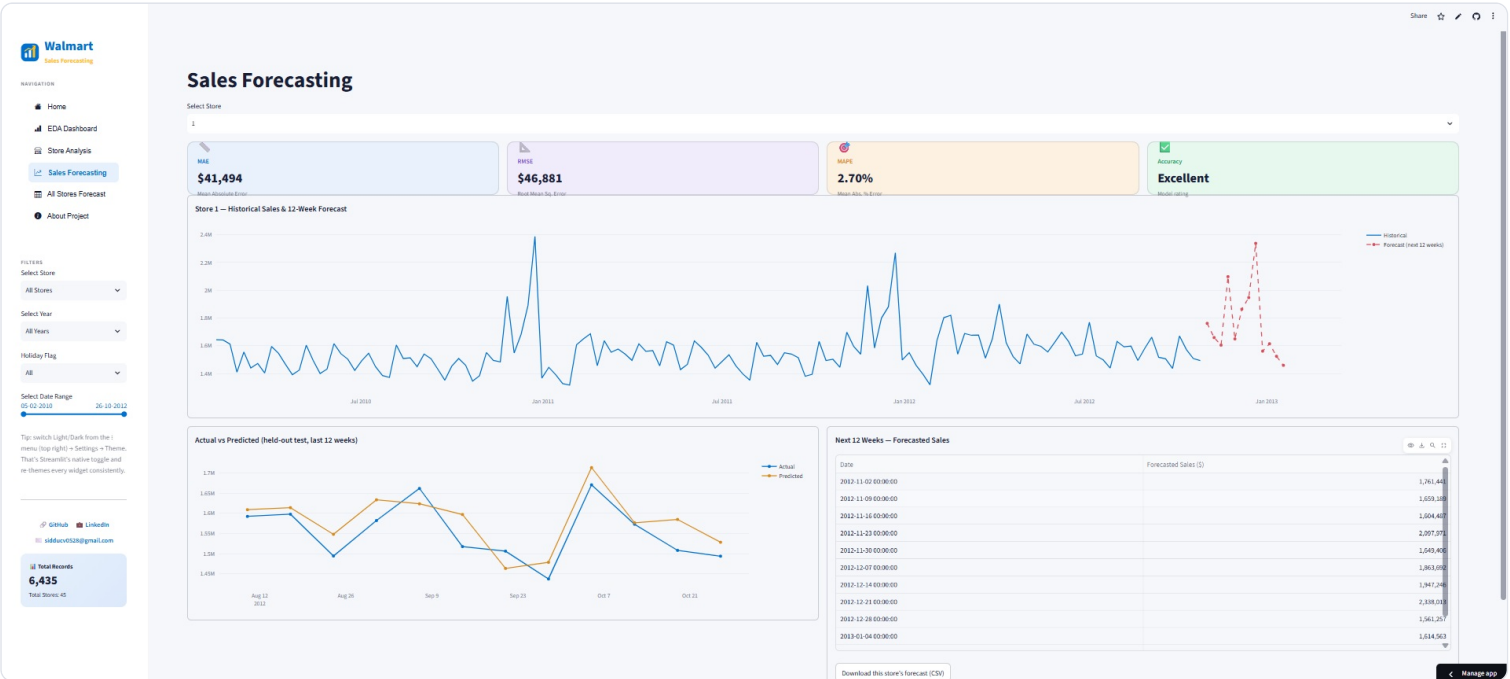
WORST STORE MAPE

14.15%

Across all 45 stores, the model averages **3.66% MAPE** — rated Excellent. **Store 37** is the best-forecasted store (1.64% MAPE), and **Store 35** is the only store outside the Excellent band, at 14.15% MAPE (still rated Good).



All Stores Forecast — MAPE by store (sorted), top 5 best/worst forecasted stores, full evaluation table



Sales Forecasting — per-store 12-week forecast against historical sales, actual vs. predicted on the held-out test window

5. Dashboard Pages

Page	What it shows
Home	KPI overview, weekly/monthly sales trends, holiday split, top 10 stores — filterable by store, year, holiday flag, and date range
EDA Dashboard	Unemployment/CPI/temperature correlation analysis, seasonality, best vs. worst store comparison
Store Analysis	Deep dive into any single store — sales rank, holiday lift, economic sensitivity, monthly seasonality
Sales Forecasting	Per-store 12-week forecast with MAE/RMSE/MAPE, actual-vs-predicted on the held-out test window, CSV download
All Stores Forecast	Model performance across all 45 stores — sorted MAPE chart, best/worst 5, full table, CSV downloads
About Project	Project summary and methodology notes



Walmart

Sales Forecasting

NAVIGATION

Home

EDA Dashboard

Store Analysis

Sales Forecasting

All Stores Forecast

About Project

FILTERS

Select Store

All Stores

Select Year

All Years

Holiday Flag

All

Select Date Range

05-02-2010 26-10-2012

Tip: switch Light/Dark from the ⚙ menu (top right) → Settings → Theme. That's Streamlit's native toggle and re-themes every widget consistently.

Share

☆

↺

⋮

About This Project

Walmart Store Sales Forecasting — Capstone Project

Objective: Forecast weekly sales for the next 12 weeks across all 45 Walmart stores and evaluate model reliability per store.

Dataset: 6,435 weekly records (45 stores × 143 weeks, Feb 2010 – Oct 2012), including `Weekly_Sales`, `Holiday_Flag`, `Temperature`, `Fuel_Price`, `CPI`, and `Unemployment`.

Model: SARIMAX(1,1,1)(1,1,1,52) — seasonal order captures the 52-week yearly cycle (holiday-driven demand spikes in November/December).

Evaluation: Last 12 weeks of each store's history held out as a test set; MAE, RMSE, and MAPE computed per store. Models with MAPE < 10% are rated Excellent, < 20% Good, else Needs Improvement.

Key EDA findings:

- Unemployment and CPI have a weak overall correlation with weekly sales, though a small subset of stores are more sensitive to local economic conditions.
- Clear seasonality: sales peak in November–December due to Thanksgiving/Christmas.
- Temperature has minimal effect on weekly sales.
- Large performance gap between best and worst performing stores, suggesting store-level inventory planning is more effective than a one-size-fits-all approach.

Built by Siddu — B.Sc. Mathematics, Statistics & Data Science.

Connect with me: [GitHub](#) · [LinkedIn](#) · [Kaggle](#) · [Email](#)

Manage app

About Project page — summary, methodology, and key findings surfaced directly in the app

6. Running & Deployment

Run Locally

```
git clone https://github.com/sidducv0528/Walmart-Sales-Forecasting.git
cd Walmart-Sales-Forecasting/walmart_deploy
pip install -r requirements.txt
streamlit run app.py
```

Why Forecasts Are Precomputed

The dashboard reads from precomputed CSVs rather than fitting SARIMA live. Fitting a seasonal model with a 52-week period across 45 stores takes several minutes — not practical to run on every page load, and would make the app unusably slow (or time out) on Streamlit Community Cloud's free tier. Instead, an offline script (`precompute.py`) fits all 45 models once and saves the results; the app simply loads and displays them, cached with `@st.cache_data` .

Refreshing the Forecast

```
cd scripts
python precompute.py
```

This regenerates the four output CSVs. Commit and push — Streamlit Cloud redeploys automatically.

7. Future Improvements

- Test true SARIMAX with exogenous regressors and compare MAPE against the current SARIMA baseline, rather than assuming the exclusion holds indefinitely.
- Per-store hyperparameter tuning (e.g. `auto_arima`) instead of one fixed order applied across all 45 stores.
- Investigate Store 35 specifically — the one clear underperformer at 14.15% MAPE.
- Automate the refresh pipeline with a scheduled job instead of a manual rerun.

GitHub: github.com/sidducv0528 | **LinkedIn:** linkedin.com/in/siddu-data | **Kaggle:** kaggle.com/sidduv0528 | **Email:** sidducv0528@gmail.com